

Sec. Engineering

Security Architecture คืออะไร:

การออกแบบและการจัดการองค์ประกอบ กระบวนการ บริการ และการควบคุมที่เหมาะสมเพื่อลดความเสี่ยงด้านความปลอดภัยที่เกี่ยวข้องกับระบบให้เหลืออยู่ในระดับที่ยอมรับได้

Security Engineering คืออะไร:

การนำการออกแบบนั้นมาปฏิบัติใช้

Security Architecture Introduction:

- เป้าหมายคือการปกป้องความลับ (Confidentiality) ความถูกต้อง (Integrity) และความพร้อมใช้งาน (Availability) ของระบบหรือธุรกิจ รวมถึงความเป็นส่วนตัว (Privacy) และหลักการสำคัญอื่นๆ
- ทำการประเมินความเสี่ยงอย่างครอบคลุมเพื่อให้ได้แนวคิดที่ถูกต้องเกี่ยวกับความเสี่ยงที่ต้องจัดการ
- เมื่อได้ระบุและประเมินความเสี่ยงแล้ว การออกแบบสถาปัตยกรรมความปลอดภัยก็สามารถเริ่มต้นได้
- **แนวทางในการจัดการความเสี่ยง (Risk treatments):**
  - หลีกเลี่ยง (Avoid)
  - โอนหรือแบ่งปัน (Transfer or share) เช่น ผ่านการประกันหรือสัญญา
  - ลดความเสี่ยง (Mitigate) เช่น ผ่านการออกแบบสถาปัตยกรรมความปลอดภัย
  - ยอมรับความเสี่ยง (Accept)
- กลยุทธ์ความปลอดภัยขององค์กรต้องสอดคล้องกับพันธกิจ เป้าหมาย และวัตถุประสงค์ รวมถึงสิ่งแวดล้อมที่เกี่ยวข้องกับข้อกำหนดและการปฏิบัติตามกฎหมาย
- ความสำเร็จในการออกแบบสถาปัตยกรรมความปลอดภัยมีแนวโน้มมากขึ้นเมื่อสิ่งที่ทำสอดคล้องกับธุรกิจ และใช้แนวทางการจัดการความเสี่ยงในการออกแบบ

Threat Modeling:

- เป็นกระบวนการที่ใช้ในการระบุภัยคุกคามและช่องโหว่ด้านความปลอดภัย รวมทั้งจัดลำดับความสำคัญของวิธีการบรรเทาภัย
- ใช้เพื่อลดความเสี่ยงและชี้แนวทางในการพัฒนาที่ปลอดภัย

Threat Modeling - STRIDE

1. **Spoofing (การปลอมแปลง)**: การโจมตีที่บุคคลหรือระบบสวมรอยเป็นอีกบุคคลหรือระบบหนึ่งโดยการปลอมแปลงข้อมูล
2. **Tampering (การดัดแปลงข้อมูล)**: การโจมตีที่ทำให้ความสมบูรณ์ของข้อมูลถูกทำลายโดยการดัดแปลงข้อมูลในทางที่เป็นอันตราย
3. **Repudiation (การปฏิเสธการกระทำ)**: เกิดขึ้นเมื่อฝ่ายใดฝ่ายหนึ่งปฏิเสธว่าไม่ได้กระทำการใด ๆ และไม่มีหลักฐานที่พิสูจน์ตรงกันข้าม
4. **Information disclosure (การเปิดเผยข้อมูล)**: การรั่วไหลของข้อมูลที่ไม่ควรเปิดเผยให้กับฝ่ายที่ไม่ได้รับอนุญาต
5. **Denial of service (การปฏิเสธการให้บริการ)**: การโจมตีที่ทำให้ระบบหรือบริการไม่สามารถใช้งานได้สำหรับผู้ใช้ที่ถูกต้อง
6. **Elevation of privilege (การยกระดับสิทธิ์)**: การโจมตีที่ทำให้ผู้ใช้ที่ไม่ได้รับอนุญาตสามารถยกระดับสิทธิ์ของตนเองไปเป็นผู้ใช้ที่มีสิทธิ์สูงกว่า (เช่น ผู้ดูแลระบบ)

Threat Modeling - DREAD

1. **Damage (ความเสียหาย)**: ภัยคุกคามสามารถก่อให้เกิดความเสียหายมากน้อยเพียงใดต่อธุรกิจของคุณ
2. **Reproducibility (การทำซ้ำได้)**: ภัยคุกคามสามารถถูกทำซ้ำหรือโจมตีซ้ำได้ง่ายเพียงใด
3. **Exploitability (การใช้ประโยชน์จากช่องโหว่)**: ความพยายามที่ต้องใช้เพื่อให้ผู้โจมตีสามารถใช้ประโยชน์จากภัยคุกคาม
4. **Affected users (ผู้ใช้ที่ได้รับผลกระทบ)**: จำนวนผู้ใช้ทั้งภายในและภายนอกองค์กรที่อาจได้รับผลกระทบจากภัยคุกคามนี้
5. **Discoverability (การค้นพบ)**: ช่องโหว่สามารถถูกค้นพบได้ง่ายเพียงใด

Threat Modeling - PASTA (7 ขั้นตอน)

1. **Define objectives (กำหนดวัตถุประสงค์)**: ระบุวัตถุประสงค์ทางธุรกิจที่สำคัญและข้อกำหนดด้านความปลอดภัย
2. **Define technical scope (กำหนดขอบเขตทางเทคนิค)**: ค้นหาและบันทึกโครงสร้างพื้นฐานและการพึ่งพาซอฟต์แวร์ทั้งหมด
3. **Application decomposition (แยกส่วนของแอปพลิเคชัน)**: ประเมินส่วนประกอบและระบุจุดเข้าทั้งหมดในแอปพลิเคชัน
4. **Threat analysis (การวิเคราะห์ภัยคุกคาม)**: ระบุและวิเคราะห์ข้อมูลภัยคุกคามจากระบบ
5. **Vulnerability analysis (การวิเคราะห์ช่องโหว่):** ระบุช่องโหว่ทั้งหมดในโค้ดของแอปพลิเคชันและประเมินความรุนแรงของแต่ละช่องโหว่ที่ตรวจพบ
6. **Attack enumeration (การระบุการโจมตี):** จำลองและวิเคราะห์การโจมตีที่อาจเกิดขึ้นตามช่องโหว่ที่พบ
7. **Risk and impact analysis (การวิเคราะห์ความเสี่ยงและผลกระทบ):** ปรับปรุงการวิเคราะห์ผลกระทบทางธุรกิจโดยพิจารณาจากการวิเคราะห์ในขั้นตอนก่อนหน้า

Threat Modeling - Trike:

Trike เป็นกรอบแนวคิดแบบครบวงจรที่เน้นไปที่การตรวจสอบความปลอดภัยจากมุมมองการจัดการความเสี่ยง โดยมีวัตถุประสงค์ในการสร้างโมเดลภัยคุกคามในลักษณะที่เชื่อถือได้และสามารถทำซ้ำได้ ทีมตรวจสอบความปลอดภัยสามารถใช้ Trike ในการอธิบาย

ลักษณะด้านความปลอดภัยของระบบได้อย่างถูกต้อง

ใน Trike จะมีการกำหนดระดับความเสี่ยงที่ยอมรับได้ให้กับทรัพย์สินแต่ละประเภท ซึ่งระดับความเสี่ยงเหล่านี้จะถูกใช้ในการกำหนดการตอบสนองต่อภัยคุกคามที่เหมาะสม

มาตรฐาน ISO/IEC 19249:2017:

คือชุดของหลักการทางสถาปัตยกรรมและการออกแบบที่สามารถใช้ในการพัฒนาผลิตภัณฑ์ ระบบ และแอปพลิเคชันให้ปลอดภัย พร้อมกับคำแนะนำในการใช้งานหลักการเหล่านั้นอย่างมีประสิทธิภาพ:

Architectural Principles

1. **Domain Separation** - การแยกโดเมนเพื่อความปลอดภัย
2. **Layering** - การแบ่งชั้นเพื่อป้องกันภัยคุกคาม
3. **Encapsulation** - การห่อหุ้มข้อมูลเพื่อป้องกันการเข้าถึงที่ไม่พึงประสงค์
4. **Redundancy** - การสำรองข้อมูลและระบบเพื่อความปลอดภัย
5. **Virtualization** - การจำลองระบบเพื่อเพิ่มความปลอดภัย

Design Principles

1. **Least Privilege** - การมอบสิทธิ์ต่ำที่สุดที่จำเป็นต่อการทำงาน
2. **Attack Surface Minimization** - การลดพื้นที่เสี่ยงต่อการถูกโจมตี
3. **Centralized Parameter Validation** - การตรวจสอบค่าพารามิเตอร์ในที่เดียว
4. **Centralized General Security Services** - การรวมบริการด้านความปลอดภัยไว้ที่ศูนย์กลาง
5. **Preparing for Error and Exception Handling** - การเตรียมรับมือกับข้อผิดพลาดและข้อยกเว้น

Secure Defaults (secure-by-default):

- ระบบควรได้รับการออกแบบให้มีความปลอดภัยสูงสุดโดยที่ผู้ใช้ไม่จำเป็นต้องเปิดใช้งานฟีเจอร์ด้านความปลอดภัยหรือกำหนดค่าความปลอดภัยด้วยตัวเอง
- การตั้งค่าเริ่มต้นของระบบควรมีการตั้งค่าที่ปลอดภัยที่สุด แม้ว่าอาจไม่ใช่การตั้งค่าที่ให้ประสิทธิภาพสูงสุดเสมอไป
- ระบบและแอปพลิเคชันควรออกแบบให้ผู้ใช้ (หรือผู้ดูแลระบบ) ต้องเลือกที่จะเปลี่ยนแปลงการตั้งค่าความปลอดภัยตามความต้องการและความเสี่ยงของธุรกิจ

Fail Security

โดยแบ่งออกเป็นสองแนวทางหลัก:

1. **Fail-open design**: ระบบยังคงเปิดให้เข้าถึงได้เมื่อเกิดข้อผิดพลาดหรือข้อยกเว้น เพื่อให้ข้อมูลที่สำคัญยังสามารถเข้าถึงได้แม้ในขณะที่ระบบมีปัญหา เช่น กรณีฉุกเฉิน
2. **Fail-secure design**: ระบบจะบล็อกการเข้าถึงตามค่าเริ่มต้นเมื่อเกิดข้อผิดพลาด เพื่อให้ความปลอดภัยมีความสำคัญเหนือความพร้อมใช้งานของระบบ

Separation of Duties (SOD)

หรือการแยกหน้าที่ โดยมีหลักการสำคัญดังนี้:

- ต้องใช้สองคนขึ้นไปหรือการกระทำและส่วนประกอบอื่นๆ เพื่อดำเนินการอย่างประสานกันในการทำงานที่เกี่ยวข้องกับความปลอดภัย
- การแบ่งกระบวนการออกเป็นหลายขั้นตอน ซึ่งดำเนินการโดยบุคคลต่างๆ หรือต้องให้บุคคลสองคนขึ้นไปทำงานร่วมกันบังคับให้ผู้ไม่ประสงค์ดีต้องรวมตัวกับคนในหลายๆ คน เพื่อกระทำการโจมตีระบบ
- กระบวนการนี้ทำให้ระบบมีความแข็งแกร่งมากขึ้นและเสี่ยงต่อความล้มเหลวน้อยลง

Keep it Simple:

- ความซับซ้อนเป็นศัตรูกับความปลอดภัย ยิ่งระบบมีความเรียบง่ายและขนาดเล็กเท่าไร ก็ยิ่งออกแบบ ประเมิน และทดสอบได้ง่ายมากขึ้น
- ถ้าระบบไม่สามารถทำให้เรียบง่ายได้ ควรแยกปัญหาออกเป็นส่วนๆ และทำให้ส่วนที่มีความเสี่ยงมากที่สุดแยกออกจากกันเพื่อให้การออกแบบปลอดภัยมากขึ้น ซึ่งจะช่วยลดความซับซ้อนในการทบทวนและทดสอบโค้ด

Trust, but (then) verify

เป็นหลักที่ใช้เพื่ออธิบายสถานการณ์ที่ต้องการการตรวจสอบและรับรองมากขึ้น:

- หลักการนี้ถูกนำไปใช้ในโลกของความปลอดภัยของข้อมูล โดยมักจะใช้กับไฟร์วอลล์เพื่อระบุตัวตน สถานที่ และคุณสมบัติอื่นๆ เพื่อยืนยันว่าผู้ใช้หรือระบบที่เข้ามานั้นน่าเชื่อถือ
- หลักการนี้ตั้งสมมติฐานว่าทุกอย่างที่อยู่หลังไฟร์วอลล์องค์กรนั้นปลอดภัย และจะมีการตรวจสอบเพื่อให้แน่ใจว่าอะไรที่ผ่านไฟร์วอลล์เข้าสู่ระบบเครือข่ายนั้นปลอดภัย
- แนวทางนี้ได้รับความนิยมน้อยลงในช่วงไม่กี่ปีที่ผ่านมา เนื่องจากมีการใช้รูปแบบความปลอดภัยแบบ "Zero Trust" มากขึ้น ซึ่งจะอธิบายในหัวข้อถัดไป

Zero Trust

ได้รับการเสนอโดย John Kindervag หลักการนี้แตกต่างจากการเชื่อถือภายในเครือข่ายแบบเก่า ซึ่งเน้นการป้องกันภัยคุกคามจากภายนอก แต่ **Zero Trust** ให้ความสำคัญกับการตรวจสอบและยืนยันทุกขั้นตอน โดยมีองค์ประกอบหลักดังนี้:

- **Always verify**: ยืนยันและอนุมัติทุกคำขอการเข้าถึงโดยใช้ข้อมูลเช่น ตัวตนของผู้ใช้ สถานที่ สุขภาพของระบบ และพฤติกรรมผู้ใช้
- **Use least privilege access**: มอบสิทธิ์ในการเข้าถึงเฉพาะส่วนที่จำเป็น และใช้หลักการ **Just in Time** (JIT) ในการให้สิทธิ์เพื่อป้องกันการเข้าถึงที่ไม่จำเป็น
- **Assume breach**: ตั้งสมมติฐานว่าระบบอาจถูกเจาะแล้ว และออกแบบมาตรการเพื่อจำกัดผลกระทบของความเสียหาย

Privacy by Design (PbD)

เป็นการออกแบบระบบที่คำนึงถึงความเป็นส่วนตัวตั้งแต่เริ่มต้น แทนที่จะมาตรวจสอบและแก้ไขในภายหลัง โดยมีหลักการสำคัญดังนี้:

- **Proactive not Reactive; Preventative not Remedial**: คาดการณ์และป้องกันเหตุการณ์ที่กระทบความเป็นส่วนตัวก่อนที่จะเกิดขึ้น แทนที่จะตรวจจับและตอบสนองเมื่อเกิดเหตุแล้ว
- **Privacy as the Default Setting**: การตั้งค่าความเป็นส่วนตัวเป็นค่าพื้นฐาน โดยจำกัดการเข้าถึงข้อมูลส่วนตัวให้เฉพาะผู้ที่จำเป็นจริง ๆ และลบข้อมูลเมื่อไม่จำเป็นต้องใช้งานแล้ว
- **Privacy Embedded into Design**: ความเป็นส่วนตัวควรเป็นฟังก์ชันหลักของระบบที่ต้องถูกพิจารณาในทุกขั้นตอนการออกแบบ

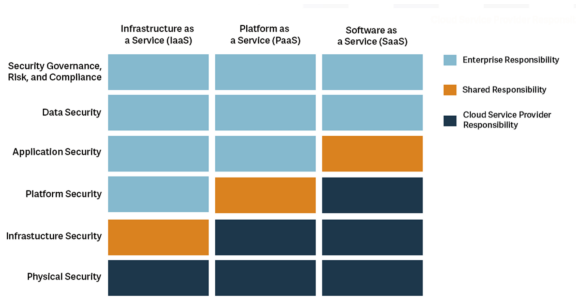
- **Full Functionality – Positive-Sum, not Zero-Sum**: กระตุ้นให้มีการออกแบบระบบที่ทำให้ทั้งความเป็นส่วนตัวและความปลอดภัยสามารถทำงานร่วมกันได้ โดยไม่ต้องมีการแลกเปลี่ยนระหว่างกัน
- **End-to-End Security – Full Lifecycle Protection**: ความมั่นคงและความเป็นส่วนตัวของข้อมูลต้องได้รับการปกป้องตลอดวงจรชีวิตของข้อมูล ตั้งแต่การสร้าง การจัดการ จนถึงการทำลายข้อมูล โดยการเข้ารหัสและการรับรองตัวตนเป็นมาตรฐานในทุกขั้นตอน
- **Visibility and Transparency – Keep it Open**: ระบบต้องเปิดเผยและโปร่งใสเพื่อให้ผู้มีส่วนเกี่ยวข้องทุกคนมั่นใจได้ว่าระบบทำงานอย่างมั่นคงและรักษาความเป็นส่วนตัวของข้อมูลตามที่ตั้งใจไว้
- **Respect for User Privacy – Keep it User-Centric**: การออกแบบระบบต้องให้ความสำคัญกับผู้ใช้เป็นหลัก โดยต้องมีการแจ้งเตือนและให้ผู้ใช้สามารถเลือกควบคุมการให้ความยินยอมในการใช้งานข้อมูลส่วนตัวได้อย่างง่ายดาย

Shared Responsibility Model:

โมเดลนี้ใช้ในระบบคลาวด์ โดยกำหนดความรับผิดชอบร่วมกันระหว่างผู้ให้บริการคลาวด์ (CSP) และลูกค้าในการดูแลความปลอดภัยของระบบคลาวด์และข้อมูล

- **ผู้ให้บริการคลาวด์ (CSP)**: มีหน้าที่ดูแลความปลอดภัยของโครงสร้างพื้นฐานคลาวด์ เช่น การรักษาความปลอดภัยทางกายภาพและแพลตฟอร์ม
- **ลูกค้า**: มีหน้าที่ดูแลความปลอดภัยของข้อมูลและการรักษาความปลอดภัยของแอปพลิเคชันที่พวกเขาใช้ในระบบคลาวด์
- แบ่งหน้าที่ชัดเจน เช่นในรูปแบบ Infrastructure as a Service (IaaS), Platform as a Service (PaaS), และ Software as a Service (SaaS)

Security Responsibility in Cloud



- แสดงตารางการแบ่งความรับผิดชอบด้านความปลอดภัยระหว่างลูกค้าและผู้ให้บริการคลาวด์ตามรูปแบบต่างๆ เช่น IaaS, PaaS, SaaS
  - **IaaS**: ลูกค้ามีหน้าที่มากที่สุดในการรักษาความปลอดภัย เช่น ความปลอดภัยของข้อมูล แอปพลิเคชัน และแพลตฟอร์ม
  - **PaaS**: ผู้ให้บริการคลาวด์มีหน้าที่เพิ่มมากขึ้นในการรักษาความปลอดภัยของแพลตฟอร์ม ส่วนลูกค้ายังคงต้องรักษาความปลอดภัยของข้อมูลและแอปพลิเคชัน
  - **SaaS**: ผู้ให้บริการคลาวด์รับผิดชอบด้านความปลอดภัยเกือบทั้งหมด ยกเว้นข้อมูลที่ลูกค้าต้องดูแล

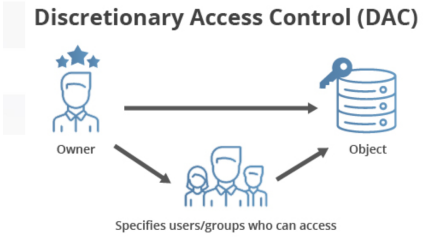
สรุปคือ ความรับผิดชอบด้านความปลอดภัยถูกแบ่งออกอย่างชัดเจนในแต่ละรูปแบบของบริการคลาวด์ ลูกค้าต้องรับผิดชอบส่วนที่อยู่ภายใต้การควบคุมของตัวเอง เช่น ข้อมูลและแอปพลิเคชัน ขณะที่ผู้ให้บริการดูแลโครงสร้างพื้นฐานและแพลตฟอร์มตามลำดับ.

Defense in Depth

หรือการป้องกันแบบหลายชั้นเป็นการใช้การควบคุมด้านความปลอดภัยที่ซ้อนทับกันหลายชั้นเพื่อให้การป้องกันโดยรวมแข็งแกร่งขึ้น:

- การใช้การควบคุมด้านความปลอดภัยหลายชั้นช่วยลดผลกระทบจากความล้มเหลวของการควบคุมใดๆ โดยไม่ส่งผลให้ระบบทั้งหมดล้มเหลว
- การจัดชั้นระบบจะช่วยแยกส่วนประกอบต่างๆ และป้องกันไม่ให้ผู้โจมตีที่สามารถเจาะผ่านชั้นหนึ่งสามารถเข้าถึงชั้นอื่นได้ง่ายๆ
- การมีการควบคุมที่ซ้อนกันยังช่วยลดความเสี่ยงจากการรั่วไหลหรือการถูกโจมตีในบางชั้นได้
- แนวคิดนี้เกี่ยวข้องกับการสันนิษฐานว่าอาจเกิดการโจมตีขึ้นแล้ว ดังนั้นจึงต้องจัดการความปลอดภัยอย่างต่อเนื่องเพื่อลดความเสี่ยง
- แนวคิดเรื่อง "Assumption of Breach" คือการเปลี่ยนมุมมองจากการเน้นปกป้องแค่ขอบเขตภายนอกของระบบ ไปสู่การจัดการป้องกันหลายชั้นที่สมดุล เพื่อให้เมื่อการควบคุมหนึ่งชั้นถูกเจาะ ระบบยังไม่ล้มเหลวทั้งหมด และมีการตรวจจับและลดผลกระทบของการโจมตีได้อย่างทันท่วงทีเทียบเท่ากับการป้องกัน

Discretionary Access Control (DAC):



DAC เป็นวิธีการควบคุมการเข้าถึงที่กำหนดโดยเจ้าของข้อมูล (Owner) ซึ่งสามารถกำหนดสิทธิ์การเข้าถึงให้กับผู้ใช้อื่นๆ ได้ โดยการควบคุมนี้มีอยู่ใน Trusted Computer System Evaluation Criteria (TCSEC) หรือที่เรียกว่า Orange Book

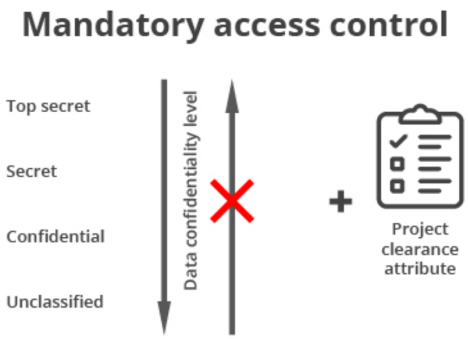
การทำงานใน DAC:

1. ผู้ใช้สร้างไฟล์และกลายเป็นเจ้าของไฟล์ หรือได้รับสิทธิ์ในการเข้าถึงไฟล์ที่มีอยู่
2. ผู้ใช้คนอื่นขอสิทธิ์ในการเข้าถึงไฟล์นี้
3. เจ้าของไฟล์สามารถให้สิทธิ์การเข้าถึงตามดุลยพินิจของตนเอง
4. หากสิทธิ์ไม่มีการขัดแย้งกับ Access Control List (ACL) ที่สร้างโดยผู้ดูแลระบบ การเข้าถึงจะได้รับการอนุญาต

ในโมเดล DAC:

- ผู้ใช้งานที่ 1 สร้างไฟล์และกลายเป็นเจ้าของไฟล์หรือได้รับสิทธิ์เข้าถึงไฟล์ที่มีอยู่
- ผู้ใช้งานที่ 2 ขอสิทธิ์เข้าถึงไฟล์นั้น
- ผู้ใช้งานที่ 1 ให้สิทธิ์เข้าถึงตามความต้องการของตน แต่ไม่สามารถให้สิทธิ์เกินกว่าที่ตนมีได้ เช่น หากผู้ใช้งานที่ 1 มีสิทธิ์อ่านได้อย่างเดียว ก็ไม่สามารถให้สิทธิ์ผู้ใช้งานที่ 2 ในการแก้ไขไฟล์นั้นได้
- หากไม่มีการขัดแย้งระหว่าง ACL (Access Control List) ที่สร้างโดยผู้ดูแลระบบและการตัดสินใจของผู้ใช้งานที่ 1 สิทธิ์ในการเข้าถึงจะถูกมอบให้ตามคำขอ

Mandatory Access Control (MAC):



MAC คือระบบการควบคุมการเข้าถึงที่ระบบปฏิบัติการกำหนดข้อจำกัดในการเข้าถึงวัตถุ (Object) หรือดำเนินการบางอย่างบนวัตถุ โดยที่นโยบายด้านความปลอดภัยจะถูกควบคุมโดยศูนย์กลาง (Security Policy Administrator) และกฎการอนุญาต (Authorization Rule) จะถูกบังคับใช้โดยเคอร์เนลของระบบปฏิบัติการ

ในกระบวนการของ MAC:

- ผู้ดูแลระบบจะกำหนดนโยบายการเข้าถึงและกำหนดคุณลักษณะด้านความปลอดภัย เช่น ระดับความลับ (Confidentiality Levels) และสิทธิ์ในการเข้าถึงสำหรับโครงการหรือทรัพยากรประเภทต่างๆ
- ผู้ดูแลระบบจะกำหนดคุณลักษณะ (Attributes) ให้กับแต่ละบุคคลหรือทรัพยากรที่ต้องการเข้าถึงข้อมูล
- เมื่อบุคคลพยายามเข้าถึงวัตถุ ระบบปฏิบัติการจะตรวจสอบคุณลักษณะด้านความปลอดภัยของบุคคลนั้นและตัดสินใจว่าจะอนุญาตให้เข้าถึงได้หรือไม่

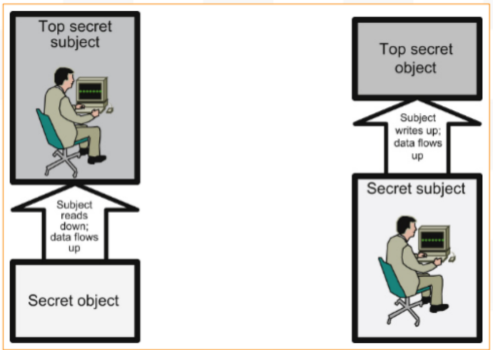
Rule-based access control (RBAC):

- การเข้าถึงถูกอนุญาตหรือปฏิเสธตามกฎที่กำหนดโดยผู้ดูแลระบบ
- คุณสมบัติการเข้าถึงถูกเก็บในรายการควบคุมการเข้าถึง (ACL) ที่เกี่ยวข้องกับแต่ละวัตถุ

Role-based access control (RBAC):

- เรียกอีกอย่างว่า Non-discretionary Access Control
- กำหนดสิทธิ์ตามบทบาทเฉพาะในองค์กร

Security Models:



- **Security Models** ใช้ในการกำหนดว่า "subject" (ซึ่งมักเป็นผู้ใช้งาน) และ "object" (ทรัพยากร) สามารถทำอะไรได้บ้างภายใต้โมเดลหรือกรอบความคิดที่กำหนด
- การจัดการความสัมพันธ์ระหว่าง subject และ object เรียกว่า access control หรือการควบคุมการเข้าถึง
- รูปภาพแสดงการจัดลำดับความสำคัญ เช่น subject ระดับลับสุดยอด (top secret) สามารถอ่านจากระดับต่ำกว่า (read down) แต่ไม่สามารถเขียนลง (write down) ข้อมูลไปยัง object ที่มีระดับต่ำกว่าได้

Fundamental Concepts of Security Models:

1. **State Machine Model**: โมเดลนี้มุ่งเน้นที่การอธิบายสถานะต่างๆ ของระบบและตรวจสอบว่าสถานะเหล่านั้นอยู่ในสถานะที่ปลอดภัย (secure states) ระบบจะเปลี่ยนจากสถานะหนึ่งไปยังอีกสถานะหนึ่ง แต่ต้องมั่นใจว่าสถานะใหม่ยังคงปลอดภัย
2. **Bell-LaPadula Model**: มุ่งเน้นในการป้องกันการรั่วไหลของข้อมูลในลำดับชั้น (hierarchical security levels) โดยใช้กฎ 2 ข้อคือ "no read up" (ผู้ใช้ในระดับต่ำไม่สามารถอ่านข้อมูลที่อยู่ในระดับสูงกว่าได้) และ "no write down" (ผู้ใช้ในระดับสูงไม่สามารถเขียนข้อมูลลงในระดับต่ำกว่าได้)
3. **Lattice-Based Access Controls**: ใช้โครงสร้าง lattice เพื่อกำหนดสิทธิ์การเข้าถึงของผู้ใช้และทรัพยากรในระบบตามลำดับชั้นที่ซับซ้อนขึ้น ผู้ใช้สามารถเข้าถึงข้อมูลได้ตามการจัดหมวดหมู่ของตัวเองใน lattice
4. **Biba Model**: เน้นเรื่องการรักษาความถูกต้องของข้อมูล (integrity) และใช้แนวคิด "no write up" และ "no read down" ซึ่งตรงข้ามกับ Bell-LaPadula model โดยป้องกันไม่ให้ผู้ใช้ที่อยู่ในระดับต่ำกว่าเขียนข้อมูลในระดับสูงกว่าเพื่อป้องกันข้อมูลไม่ถูกแก้ไขโดยไม่ได้รับอนุญาต
5. **Clark-Wilson Model**: เน้นการป้องกันการทุจริตหรือการทำให้ข้อมูลผิดเพี้ยน โดยกำหนดกฎและขั้นตอนการทำงาน (well-formed transactions) เพื่อให้มั่นใจว่าข้อมูลจะถูกแก้ไขหรือเปลี่ยนแปลงได้โดยผ่านกระบวนการที่เชื่อถือได้เท่านั้น
6. **Information Flow Model**: มุ่งเน้นการควบคุมการไหลของข้อมูลจากแหล่งข้อมูลหนึ่งไปยังอีกแหล่งข้อมูลหนึ่งเพื่อป้องกันการรั่วไหลของข้อมูลลับหรือสำคัญ การควบคุมนี้สามารถเกิดขึ้นได้ทั้งในระดับข้อมูลและระบบ
7. **Brewer and Nash Model (Chinese Wall Model)**: ป้องกันการขัดกันของผลประโยชน์ (conflict of interest) โดยไม่อนุญาตให้ผู้ใช้เข้าถึงข้อมูลที่อยู่ในกลุ่มที่ขัดกันภายในองค์กร ซึ่งเหมาะสมกับธุรกิจที่มีการจัดการข้อมูลลูกค้าที่อาจขัดผลประโยชน์กันได้ เช่น การธนาคาร หรือสำนักงานกฎหมาย
8. **Take-Grant Model**: ใช้การจัดการสิทธิ์โดยให้สิทธิ์ในการโอนสิทธิ์หรือการเข้าถึงทรัพยากรจากผู้ใช้หนึ่งไปยังอีกผู้ใช้หนึ่งได้ตามกฎที่กำหนด
9. **Access Control Matrix**: ตารางที่ใช้กำหนดสิทธิ์ของผู้ใช้แต่ละรายในการเข้าถึงทรัพยากรต่างๆ ในระบบโดยชัดเจน มีการกำหนดว่าใครสามารถทำอะไรกับทรัพยากรไหนได้บ้าง
10. **Zachman Framework for Enterprise Architecture**: โมเดลนี้เน้นที่การออกแบบโครงสร้างพื้นฐานในองค์กรโดยใช้การจัดการเชิงสถาปัตยกรรมเพื่อให้เกิดความปลอดภัยและมีประสิทธิภาพ
11. **Graham-Denning Model**: มุ่งเน้นไปที่การจัดการและควบคุมการเข้าถึงโดยกำหนดให้มีกฎในการสร้าง ลบ และเปลี่ยนแปลงสิทธิ์ของผู้ใช้ในระบบ

State Machine Model

คือโมเดลความปลอดภัยที่มุ่งเน้นการตรวจสอบสถานะของระบบเพื่อให้มั่นใจว่าระบบยังคงมีความปลอดภัย สถานะ (State) ในระบบนี้จะประกอบด้วยสิทธิ์ในการเข้าถึงทั้งหมดที่มีอยู่และตัวอย่างของผู้ใช้งาน (subjects) ที่เข้าถึงวัตถุ (objects) หากผู้ใช้สามารถเข้าถึงวัตถุได้ตามนโยบายความปลอดภัย ระบบจะถือว่าปลอดภัย

Key Concepts:

- **Finite State Machine (FSM)**: ระบบจะมีสถานะที่จำกัด (finite) และเปลี่ยนแปลงตามการกระทำ

- **State Transition**: การเปลี่ยนแปลงระหว่างสถานะต่างๆ จะต้องมั่นใจว่าอยู่ในสถานะที่ปลอดภัย (secure states)
- **Secure State Machine**: ระบบจะปลอดภัยไม่ว่าจะอยู่ในสถานะใดก็ตาม

โมเดลนี้เป็นพื้นฐานที่สำคัญสำหรับโมเดลความปลอดภัยอื่นๆ อีกมากมาย

Bell-LaPadula Model ****

เน้นความลับ (confidentiality) ของข้อมูล โดยถูกพัฒนาขึ้นเพื่อใช้งานกับกระทรวงกลาโหมสหรัฐฯ

กฎการเข้าถึงข้อมูล:

- **Simple Security Property**: ห้าม "อ่านขึ้น" (No read up) เช่น ระดับข้อมูลล่าง (Confidential) ห้ามอ่านข้อมูลระดับสูง (Secret)
- **Star Security Property**: ห้าม "เขียนลง" (No write down) เช่น ข้อมูลระดับสูง (Top Secret) ห้ามเขียนลงไปในระดับต่ำกว่า (Secret หรือ Confidential)

กฎของการจัดการป้าย (Labeling Rules):

- **Strong Tranquility Property**: ป้ายความปลอดภัย (security labels) จะไม่เปลี่ยนแปลงขณะที่ระบบกำลังทำงาน
- **Weak Tranquility Property**: ป้ายความปลอดภัยจะไม่เปลี่ยนแปลงในลักษณะที่ขัดแย้งกับคุณสมบัติด้านความปลอดภัยที่กำหนด

Biba Model *****

เป็นโมเดลที่ถูกพัฒนาต่อจาก Bell-LaPadula โดยเน้นที่ **ความสมบูรณ์ของข้อมูล (Integrity)** มากกว่าความลับของข้อมูล

กฎหลัก 2 ข้อในโมเดลนี้คือ:

- **Simple Integrity Axiom**: ห้าม "อ่านลง" (No read down) เช่น ข้อมูลระดับสูง (Top Secret) ห้ามอ่านข้อมูลที่อยู่ในระดับต่ำกว่า (Secret)
- **Star Integrity Axiom**: ห้าม "เขียนขึ้น" (No write up) เช่น ข้อมูลระดับต่ำ (Unclassified) ห้ามเขียนขึ้นไปในระดับสูงกว่า (Confidential)

Lattice-Based Access Controls

เป็นโมเดลการควบคุมการเข้าถึงสำหรับสภาพแวดล้อมที่ซับซ้อน ซึ่งสำหรับทุกความสัมพันธ์ระหว่างผู้ใช้ (subject) และวัตถุ (object) จะมีการกำหนดขอบเขตการเข้าถึงสูงสุด (Upper Bound) และต่ำสุด (Lower Bound) ที่ระบบจะบังคับใช้

- **Least Upper Bound (LUB)** และ **Greatest Lower Bound (GLB)** จะถูกใช้เพื่อกำหนดขอบเขตการเข้าถึง โดยอิงตามตำแหน่งใน lattice (โครงสร้างซ้อนกัน)
- โมเดลนี้รวมความปลอดภัยทั้งแบบหลายระดับ (multilevel) และหลายทาง (multilateral)

Clark-Wilson Model

เป็นโมเดลด้านความสมบูรณ์ของข้อมูล (integrity) ที่เน้นการตรวจสอบว่าการเข้าถึงข้อมูลของผู้ใช้ต้องผ่านโปรแกรมที่มีการควบคุมและมีขอบเขตที่ชัดเจน โดยมีแนวคิดสำคัญสองข้อคือ

1. **Well-Formed Transactions** - การทำธุรกรรมต้องเกิดขึ้นผ่านโปรแกรมที่ถูกออกแบบให้ถูกต้อง และมีการควบคุมด้วยสามส่วนหลัก (ผู้ใช้, การทำธุรกรรม, ข้อมูลที่ต้องการความสมบูรณ์) เพื่อให้แน่ใจว่าข้อมูลถูกเก็บรักษาในสถานะที่ถูกต้อง
2. **Separation of Duties** - ผู้ใช้ที่ได้รับอนุญาตต้องไม่เปลี่ยนแปลงข้อมูลในวิธีที่ไม่เหมาะสม

Information Flow Model

เป็นโมเดลที่จัดการกับข้อมูลโดยแบ่งข้อมูลออกเป็น compartment หรือส่วนย่อยๆ โดยข้อมูลจะถูกจัดประเภทตามสองปัจจัยหลัก ได้แก่ การจำแนกข้อมูล (classification) และความจำเป็นในการรู้ (need to know)

- **การแบ่ง compartment**: ข้อมูลถูกแยกตามการจำแนกและความจำเป็นในการรู้ ข้อมูลจึงถูกป้องกันจากการเข้าถึงที่ไม่ได้รับอนุญาต
- **Subject clearance**: การอนุญาตให้เข้าถึงข้อมูล (clearance) ต้องมีระดับสูงกว่าหรือเท่ากับการจำแนกข้อมูลของ object และ subject จะต้องมีสิทธิ์ใน profile ที่สอดคล้องกับหมวดหมู่ที่กำหนดในป้ายชื่อของ object

Brewer and Nash Model (หรือ Chinese Wall Model)

เป็นโมเดลที่ออกแบบมาเพื่อหลีกเลี่ยงปัญหาความขัดแย้งทางผลประโยชน์ โดยโมเดลนี้จะป้องกันไม่ให้บุคคล เช่น ที่ปรึกษา เข้าถึงข้อมูลหรือกลุ่มข้อมูลที่ขัดแย้งกัน

- โมเดลนี้มีการควบคุมการเข้าถึงที่สามารถเปลี่ยนแปลงได้ตามพฤติกรรมก่อนหน้านี้ของผู้ใช้งาน
- กำหนดว่าผู้ใช้งานสามารถเขียนข้อมูลไปยัง object ได้ก็ต่อเมื่อยังไม่ได้อ่านข้อมูลจาก object ที่ขัดแย้งกันใน data set อื่น
- ออกแบบมาเพื่อจัดการกับความเสี่ยงจากการว่าจ้างที่ปรึกษาที่ทำงานในอุตสาหกรรมการเงินหรือธนาคาร

Noninterference Model

คือโมเดลที่ออกแบบมาเพื่อรับประกันว่าการกระทำที่เกิดขึ้นในระดับความปลอดภัยที่สูงขึ้นจะไม่ส่งผลกระทบหรือรบกวนการกระทำที่เกิดขึ้นในระดับที่ต่ำกว่า โมเดลนี้ไม่สนใจเกี่ยวกับการไหลของข้อมูล แต่จะเน้นว่าผู้ใช้งานเข้าใจสถานะของระบบอย่างไร

- เน้นการป้องกันไม่ให้เกิดการรบกวนจากการกระทำในระดับต่างๆ ของระบบ
- แก้ปัญหาการโจมตีแบบ **inference attack** ที่เกิดขึ้นเมื่อผู้ใช้งานสามารถอนุมานบางสิ่งบางอย่างจากข้อมูลที่เขาไม่มีสิทธิ์เข้าถึง
- ป้องกัน **Covert Channel** ซึ่งเป็นการละเมิดนโยบายที่ถูกซ่อนจากผู้ดูแลระบบ

Covert Channel

แบ่งเป็น 2 ประเภทหลัก:

1. **Covert Storage Channel**
    - เป็นช่องทางที่ถ่ายทอดข้อมูลโดยการเขียนข้อมูลไปยังพื้นที่จัดเก็บที่สามารถถูกอ่านโดยอีกกระบวนการหนึ่ง
    - การเขียนและอ่านนี้สามารถเป็นทางตรงหรือทางอ้อม และเกี่ยวข้องกับการแชร์ทรัพยากรที่ถูกใช้โดยบุคคลที่มีระดับความปลอดภัยต่างกัน
2. **Covert Timing Channel**
    - เป็นช่องทางที่ถ่ายทอดข้อมูลโดยการปรับแต่งเวลาของระบบหรือส่วนประกอบต่างๆ ให้เกิดการคาดการณ์ที่แน่นอน
    - เช่น การใช้ทรัพยากรของระบบในลักษณะที่ทำให้เวลาตอบสนองที่ถูกสังเกตโดยกระบวนการที่สองเปลี่ยนแปลง

Take-Grant Model

โมเดลนี้กำหนดกฎการโต้ตอบระหว่าง subject และ object รวมถึงสิทธิ์ที่ subject สามารถมอบให้กับ subject อื่นได้ โดยมีสิทธิ์ 2 ประเภทหลักคือ **take** และ **grant**

- **Take rule**: อนุญาตให้ subject นำสิทธิ์จาก object อื่นได้

- **Grant rule**: อนุญาตให้ subject มอบสิทธิ์ของตัวเองให้กับ object อื่นได้
- **Create rule**: อนุญาตให้ subject สร้าง object ใหม่ได้
- **Remove rule**: อนุญาตให้ subject ลบสิทธิ์ที่มีอยู่ใน object ได้

Access Control Matrix

Access Control Matrix เป็นตารางที่ใช้บ่อยในระบบปฏิบัติการ (OS) และแอปพลิเคชันที่กำหนดสิทธิ์การเข้าถึงระหว่าง subject (ผู้ใช้) และ object (ไฟล์หรือทรัพยากร) โดยในตารางนี้จะระบุว่าผู้ใช้แต่ละคนสามารถเข้าถึงไฟล์แต่ละไฟล์ได้อย่างไร เช่น อ่านอย่างเดียว, เขียนอย่างเดียว, หรือเข้าถึงแบบ Full Control เป็นต้น

Graham-Denning Model

โมเดล Graham-Denning มีการกำหนดชุดสิทธิขั้นพื้นฐานในรูปแบบของคำสั่งที่ subject แต่ละคนสามารถกระทำต่อ object ได้ โดยโมเดลนี้ประกอบด้วยสามส่วนหลัก ๆ ได้แก่ object, subject และกฎ (rules) ซึ่งในโมเดลนี้จะมีการเน้นไปที่กฎ 8 ข้อ ได้แก่:

1. Transfer Access - การโอนสิทธิการเข้าถึง
2. Grant Access - การมอบสิทธิการเข้าถึง
3. Delete Access - การลบสิทธิการเข้าถึง
4. Read Object - การอ่าน object
5. Create Object - การสร้าง object
6. Destroy Object - การทำลาย object
7. Create Subject - การสร้าง subject
8. Destroy Subject - การทำลาย subject